Response Cache for an LLM Service

Background

An LLM chatbot service receives thousands of prompts every minute, and many of them are repeats ("What is the capital of France?", "Summarize this policy", and so on). Generating a response with the LLM is expensive. To save cost, the service keeps a **response cache**: once a response for a prompt has been generated, it is stored, and the next time the same prompt arrives, the stored response is returned without running the model again.

Each prompt is first reduced to a **fingerprint**, a positive integer between 1 and 10^7. Two prompts with the same fingerprint are treated as the same prompt. Your job is to build the index that records which fingerprints have a cached response.

The engineering team wants this index to be a **hash table with separate chaining**. They also want the index to report how much work each lookup takes, so that they can monitor whether the cache is becoming slow and which part of the table is overloaded.

The Cache Index

The index has a fixed number of buckets `M`, numbered `0` to `M-1`. Each bucket holds a linked list (chain) of fingerprints. The number of buckets never changes, and fingerprints are never removed.

- **Hash function.** Fingerprint `f` belongs to bucket `h(f) = f mod M`. You must use exactly this hash function. The comparison counts, the `DUMP` output and the `MOSTPOP` answer all depend on which bucket each fingerprint lands in, so a different hash function will produce different (wrong) output.
- **New entries go to the front.** A newly cached fingerprint is always added at the **head** of its bucket's chain. Recently cached prompts are more likely to be requested again, so keeping them near the front makes their lookups faster.
- **Comparisons.** Searching for `f` walks its bucket's chain from the head and compares `f` with each fingerprint it visits. If `f` is the `c`-th fingerprint in the chain, the search makes `c` comparisons. If `f` is not in the chain, the search makes as many comparisons as there are fingerprints in the chain (0 for an empty bucket).

Requests

The service sends the index a sequence of requests. Process them in order and print one line for each.

`STORE f`: a response for fingerprint `f` has just been generated and should be cached.

- If `f` is already cached, print `DUPLICATE f`. Nothing changes.
- Otherwise, add `f` at the head of its chain and print `STORED f`.

`LOOKUP f`: a prompt with fingerprint `f` has arrived.

- If `f` is cached, print `HIT c`. Otherwise, print `MISS c`. In both cases, `c` is the number of comparisons made.
- A lookup does **not** change the order of the chain.

`MOSTPOP`: the monitoring dashboard asks for the most populated bucket.

- Print `MOSTPOP b L`, where `b` is the bucket with the longest chain and `L` is its length. If several buckets tie, report the one with the smallest index. If the cache is empty, print `MOSTPOP 0 0`.

`DUMP b`: an engineer wants to inspect bucket `b` (`0 ≤ b < M`).

- Print the fingerprints in bucket `b` from head to tail, separated by single spaces. If the bucket is empty, print `EMPTY`.

Input Format

The first line contains two integers `M` and `Q`: the number of buckets and the number of requests.

Each of the next `Q` lines contains one request in one of the formats above.

Output Format

Print exactly one line for each request, in the order the requests are given, using the formats described above.

Constraints

- `1 ≤ M ≤ 10000`
- `1 ≤ Q ≤ 100000`
- `1 ≤ f ≤ 10^7`
- No chain ever holds more than 1000 fingerprints.
- At most 100 `MOSTPOP` requests and at most 100 `DUMP` requests appear in a test.
- Time limit: 2 seconds per test. Memory limit: 256 MB.
- Implement the chains as linked lists. Do not use any other data structure or library to store or find fingerprints.

Starter Code

You are given the C file `llm_cache_starter.c`. It already contains the data structures, the hash function `hash_fingerprint()` and a complete `main()` that reads the input and prints every response in the required format. Do not modify `main()`, the hash function or the function signatures.

Fill in the bodies of the following six functions. Each one is marked `TODO` in the file, and its comment describes exactly what it must do.

```
HashTable *create_table(int m);
int  search(HashTable *t, int fp, int *comparisons);
int  store(HashTable *t, int fp);
void mostpop(HashTable *t, int *bucket, int *length);
void dump_bucket(HashTable *t, int b);
void free_table(HashTable *t);
```

The file compiles as it is, so you can build and test after implementing each function:

```
gcc -O2 -Wall -o llm_cache llm_cache_starter.c
./llm_cache < input.txt
```

Sample

Input

```
5 13
STORE 12
STORE 7
STORE 22
STORE 9
DUMP 2
LOOKUP 12
LOOKUP 7
LOOKUP 17
LOOKUP 3
STORE 7
STORE 14
MOSTPOP
DUMP 3
```

Output

```
STORED 12
STORED 7
STORED 22
STORED 9
22 7 12
HIT 3
HIT 2
MISS 3
MISS 0
DUPLICATE 7
STORED 14
MOSTPOP 2 3
EMPTY
```

Explanation

- With `M = 5`, fingerprints 12, 7 and 22 all belong to bucket 2. Each is added at the head, so the chain reads `22 7 12`. Fingerprint 9 goes to bucket 4.
- Finding 12 takes 3 comparisons (22, 7, 12), and finding 7 takes 2.
- 17 also belongs to bucket 2. All 3 fingerprints are compared and none match, so the result is `MISS 3`. Bucket 3 is empty, so `LOOKUP 3` makes 0 comparisons.
- 7 is already cached, so storing it again prints `DUPLICATE 7`.
- 14 goes to the head of bucket 4, which now reads `14 9`. Bucket 2 (3 fingerprints) is still the longest, so `MOSTPOP` prints `MOSTPOP 2 3`.

Hints

- Inserting at the head of a linked list takes constant time and does not need a loop.